

Threat-Priority Scheduling — Brief

For the RL lane. Written 2026-09-11. One change: the scheduler accepts a priority input and learns to use it. No environment change, nothing frozen moves.

Owner: Aamir. PDF: `python -m scripts.md2pdf docs/project/THREAT_WEIGHTING_BRIEF.md`.

Outranked by `SIH26055_PROBLEM_STATEMENT.md`, `DECISIONS.md`, `ENVIRONMENT_SPEC.md`, `EVALUATION.md`.
Where this and those disagree, those win.

0. What is ours and what is not

Threat prioritisation is not our idea. It is in the problem statement. Any team that reads the Background will propose it. Stating it as our contribution invites the obvious question and we lose the slide.

What is measurably ours is narrower and stronger. The word *threat* appears in the PS **exactly once** — in the Background, as the motivation. The **Detailed Description**, which is the actual specification, never says it again: its nine figures of merit have no threat term and its objective sentence is *"minimize intercept time and ensure a high interception rate."* `EVALUATION.md` has **0 occurrences** (both counted 2026-09-11 by `grep -c -i threat`).

DRDO names the failure in paragraph one and then measures you on something else. Every team will optimise the specified metrics and inherit the failure the Background describes. Three things close that loop, and all three are ours:

	Ours	Why it is not obvious
The negative result	Threat cannot be computed at the scan-scheduler layer. Best band-level AUC 0.617 ; "hard to intercept" 0.514 ; "busiest bands" inverted . §2 has the mechanism.	The default assumption — "the agent will learn which emitters matter" — is false here, and we are the only ones who will have measured it. It makes the architecture <i>forced</i> , not chosen.
The interface	One priority vector <code>p</code> , default uniform. An operator and a downstream categoriser drive the same code path .	It is what makes autonomous-by-default and operator-steerable one system instead of a mode switch.
The metric	The §4 scorecard reported split by threat class .	Nobody's scorecard has this, including the PS's own. Without it the Background sentence is unmeasurable and the claim is decoration.

On autonomy, precisely. The system is autonomous: the priority arrives without a human in the normal case, from the processing unit's threat library. That library is a **lookup on a fingerprint, not an inference engine** — analysts assign the ratings offline, the system matches PDWs against the table at runtime. It is why it works where §2 failed: it reads the full PDW stream, which we deliberately do not have (D12/D19 scoped deinterleaving out long before this idea existed).

We are not claiming to have built that categoriser, and we are not claiming the scheduler infers threat — §2 is us proving it cannot. Nobody building a scan scheduler builds the whole radar. The claim is that the scheduler is the one box nobody ever wired the library to, and that a human can step into the same input when they know something the library does not.

The one-line version: **the doctrine is supplied offline; the execution is autonomous.**

1. The gap

A modern ES system already knows which emitters are dangerous. The threat library sits in the processing unit; the operator has a mission brief. **None of it reaches the scan scheduler.** The scheduler sweeps a schedule computed before the mission and spends the same 50 ms on a weather radar as on a fire-control radar.

The problem statement says exactly this:

"Open loop strategies focus only on this requirement and may lose time to nonthreatening emitters by not giving time to new or threatening ones."

We build the scheduler that gets told.

2. What we are and are not claiming

We do not define threat, and we do not infer it. Threat is doctrine plus mission context — a fire-control radar is dangerous because of what it is attached to, not because of anything in its pulse train. It is supplied to us.

That is not a hedge; it is measured. Two attempts and both failed:

Attempt	Result
Infer threat from what the receiver observes	Best band-level AUC 0.617 across hit rate, mean/max level, level std, intermittency (772 occupied band-pairs). Mechanism: $S = \max$ over contributors (D25) — a 0.3 kW fire-control radar sharing a band with a 750 kW weather radar never reaches that band's statistics.
Use "hard to intercept" as a proxy	AUC 0.514 for predicting HIGH threat. Difficulty is driven by transmit power ($r = +0.485$), not threat. Not by range ($r = -0.051$), which was the first alternative ruled out.
"Threat = the busiest bands"	Inverted. Threat fraction by band-density quartile runs 0.379 / 0.227 / 0.116 / 0.091, sparsest to busiest.

So threat weighting always comes from outside the scheduler. Claiming a self-categorising system would be false and a domain reviewer will find it in one question.

Where it comes from in a real system — ADITI 4.0's own decomposition (transcript p.2): *"the system basically consists of antenna, receiver, processing unit, database..."* Categorisation is a processing-unit function over the full PDW stream. We are a different box, and the thing ADITI asks for between them is the *"feedback based decision making mechanism"* — which is this.

3. What gets built

The scheduler takes a per-band priority vector $p \in \mathbb{R}^3$.

$p = 1$ everywhere -> the plain PS objective. The default. Nothing changes.
 p weighted -> the scheduler re-plans around what matters this mission.

p has two sources and they are the same code: a human operator, or a downstream categoriser. That is what makes "autonomous by default, supervised when a human has something to add" one system rather than two.

How it enters:

1. p is concatenated to the observation: **146** → **182**.
2. During training p is **sampled per episode** — sometimes uniform, sometimes weighted — so the policy learns to use it rather than memorising one setting.
3. The reward multiplies per-emitter discovery credit by p[band]. Weight **discovery only**, not per-slot occupancy: a weighted per-slot term pays repeatedly for camping on one emitter, which is D53's failure mode with a new coefficient on it. **newly** fires once per (emitter, band) pair (D51), which is the right granularity.

This is a retrain and every existing checkpoint dies, as they did at D49 and again at D55. That is the cost; it is not a bolt-on.

3.1 Why priority is a post-discovery problem

The PS separates two categories in one sentence: time should go to "*new or threatening*" emitters. **New** means not yet found. **Threatening** means already found and identified. You cannot know an emitter is a fire-control radar until you have intercepted it, so threat priority is inherently about *what you do with airtime after discovery*.

That is not a hole in the idea, it is the mechanism, and it is the reason this fits the machinery we already have: "**new vs threatening**" is **D14's explore/exploit tension with a threat term on the exploit side**. The scheduler was always trading breadth against depth; threat is what tells it which depth is worth buying.

So p is loaded at episode start and it may also move during the episode as emitters are identified. Both are the same interface.

One bounded question for Aamir, and it is the only open item here. The in-episode update means the observation learns something truth-side — that the emitter just intercepted is HIGH. **D29 says the reward may read truth and the observation may not.** The case for allowing it: this is not leakage but a stand-in for the processing-unit function we abstracted away, and it reveals only what a real ES system genuinely knows at that instant. Episode-start p has no such question and can be built today.

4. The demo, which is the deliverable

Operator marks bands as priority. The waterfall re-plans in front of the judge. The metrics move.

A slide describing this architecture is worth very little. Thirty seconds of it running is the thing they remember.

5. How it gets scored, and the example threat library

Report the EVALUATION.md §4 table **split by threat class**, alongside the pooled table — never instead of it. The nine PS figures of merit are reported unchanged.

For the demo and the scoring we need *an* instance of a threat library. `metadata/transmitters/*/attrs['function']` names all 68 emitter types, classified the way a real RWR would:

class	contents	n	share
HIGH	fire control, engagement, missile guidance, counter-battery (it is locating <i>your</i> guns), LPI	656	38.5%
MEDIUM	air-defence search, early warning, surveillance, maritime patrol	775	45.5%
LOW	weather, marine navigation, airport/ground movement, SAR, ground-penetrating	273	16.0%

Label this as an example, not as our threat model. It is domain knowledge applied to a dataset field, it goes in code before any scheduler is scored against it, and it never changes afterwards. A weighting defined once results are visible is not a weighting — this repository has withdrawn four figures for exactly that failure (D33, D41, and two in the 2026-09-03 audit).

6. Order of work

Ordered so the cheapest thing that could kill the idea runs first. Steps 0 and 1 need no agent and no checkpoint — `runs/checkpoints/` is empty here and every checkpoint died at D49/D55 anyway.

	Do	Cost	Kills the idea if...
0	P2 — the band-mixing rate. For every occupied band across the 47 configs, the threat classes of its contributors.	One hour, HDF5 only.	Most occupied bands hold both a HIGH and a LOW emitter. Band priority then cannot discriminate. Go to §7.3 before going further.
1	P1 — is the split coarse? The 38.5% HIGH share, and whether a narrower top class (fire control + missile guidance only) is usefully smaller.	One hour, HDF5 only.	Nothing narrows below roughly a third. <code>p</code> is then near-uniform in practice.
2	Score any RL-lane checkpoint split by the §5 classes	An afternoon, no retrain, needs a checkpoint from the RL lane.	— Gives a slide either way: does the scheduler already favour HIGH emitters by accident?
3	Add <code>p</code> to the observation, sample it in training, weight discovery credit by it — and train the <code>p</code> = uniform control arm alongside (P5)	The retrain.	—

	Do	Cost	Kills the idea if...
4	P4 — the permutation ablation. Freeze the checkpoint, shuffle <code>p</code> , re-run.	Minutes, once trained.	Behaviour does not change. The agent ignored <code>p</code> and the feature is dead.
5	Selection and scoring: training half only (D60), <code>rfenv/selection.py</code> (D61), D47's rule	Existing machinery.	—
6	Build the demo	The deliverable.	—

Steps 0 and 1 are the gate. **Do not spend a retrain before they pass** — and if they fail, §7.3 says what to change rather than what to abandon.

7. Known problems, and where the idea gets better

None of the following is measured. They are the honest failure modes of §3, written down before anyone builds it, because the refinements that answer them are what turn a reasonable idea into a strong one. Each row names the cheapest thing that would settle it.

7.1 The blunt-knob problems — these can be settled today, with no agent

#	Problem	Why it could be fatal	Cheapest test
P1	The HIGH class is 38.5% of emitters.	If two in five emitters are "important", prioritising them barely narrows the search and <code>p</code> is close to uniform in practice. The idea dies quietly — it trains, it just does nothing.	Count it. Then re-split: does a finer top class (fire control + missile guidance only, excluding early warning) give a usefully small share?
P2	Priority is per band; threat is per emitter. Bands overlap by half (D3), most emitters straddle two, and <code>S = max</code> (D25).	If most occupied bands hold both a HIGH and a LOW emitter, band priority cannot discriminate and there is nothing for the agent to learn. This is the single most likely way the idea fails.	Measure the band-mixing rate across the 47 configs: for each occupied band, the threat classes of its contributors. One hour on the HDF5 files.
P3	Thin scoring. The §5 split reports per class, on 12 validation configs (D60).	If HIGH-class intercepts per episode are few, the split cannot separate schedulers and the headline metric is theatre.	Count HIGH-class intercepts per episode under rung 5. If the count is small, report pooled-with-effect-size instead of a split table.

P1 and P2 together decide whether this is worth a retrain. Run them first. If P2 comes back "most bands are mixed", the fix is not to abandon the idea but to change what `p` encodes — see 7.3.

7.2 The training problems — need the RL lane

#	Problem	Why it could be fatal	Cheapest test
P4	The agent ignores $\mathbf{p}$. The standard failure of a conditioned policy: if the priority term is weak against the other reward terms, the policy averages over it.	You ship a scheduler with 36 extra inputs and identical behaviour.	The permutation ablation. Freeze the checkpoint, shuffle $\mathbf{p}$, re-run. If behaviour does not change, the feature is dead. This is also the cleanest slide in the deck if it <i>does</i> change.
P5	No control arm. CLAUDE.md: no RL result in this repository is currently clean.	"Threat weighting helps" is unfalsifiable without an identically-trained $\mathbf{p} = \text{uniform}$ arm on the same split.	Train the uniform arm alongside. Non-negotiable, not optional.
P6	$\mathbf{p}$ needs scaling. D55 found two of three observation blocks living in the bottom tenth of their range.	A priority block on a different scale to its 146 neighbours trains badly for reasons that have nothing to do with the idea.	Apply D55's reasoning: state the units, put 1.0 at "ordinary".

7.3 Refinements — what makes the edge better, not just survivable

Each of these is a *response to a row above*, which is why they are worth more than a longer feature list. All are unmeasured candidates, not decisions.

- **Graded priority instead of three buckets (answers P1).** $\mathbf{p}$ is already a real vector; nothing forces it to take three values. A continuous rating per radar model is closer to what a real threat library holds anyway, and it removes the arbitrariness of the class boundaries.
- **Split HIGH by what the radar is *doing* (answers P1).** Real RWRs already distinguish a search illumination from a lock-on, and treat them as different alarms. "Something is looking for me" and "something is guiding a weapon at me" collapsing into one class is our simplification, not the domain's.
- **Make $\mathbf{p}$ an expected threat per band, not a max (answers P2).** If a band holds a HIGH and a LOW emitter, its priority is the threat weighted by each contributor's share of the band. That is a strictly better-posed quantity than "the band contains something scary" and it degrades gracefully as mixing rises.
- **Threat $\times$ staleness (answers P2 and P4).** You care most about a dangerous emitter you have *not* looked at recently. This is D14's explore/exploit tension expressed inside the priority itself, and it gives the agent a signal that varies during an episode rather than a constant.
- **Cite doctrine for the classification, not our judgement (answers a credibility risk).** §5 is currently our reading of 68 radar names. A domain reviewer who disagrees with one assignment can unpick the demo. An external source for the ratings removes that.

7.4 What we are choosing not to fix

Situational threat. A search radar that has just acquired you is more dangerous than the same radar sweeping, and our library is static. That is real, it is what "cognitive" EW eventually means, and it needs mode recognition we do not have (D12/D19). Name it as future work; do not build it.

8. What does not move

`rfenv/constants.py` is untouched. D42 puts reward candidates and observation extensions deliberately outside the freeze list; D57 lifted D29's cap of three candidates. No gate re-runs, no baseline re-runs. `DEFAULT_REWARD` stays `reward_balance` as the control arm.